TP n°2

V. Ledda

2 octobre 2026

---

1 Régression polynômiale.

1.1 Cache cache

J'ai caché un code secret à 4 chiffres dans le fichier `secret.csv` (disponible sur `Moodle`).
Pour ce faire, j'ai généré 200 nombres aléatoires entre -10 et 5, pour obtenir un vecteur x. Ensuite, j'ai calculé y par $y = ax^3 + by^2 + cy + d$ où $abcd$ est le code secret. Enfin, avant d'enregistrer le fichier, j'ai bruité les valeurs de y.
Le code est le suivant.

```
import numpy as np
import matplotlib.pyplot as plt
import csv
f = lambda x:a*x**3+b*x**2+c*x+d
x = 15*np.random.rand(1,n)-10*np.ones((1,n)) #x\in [-10;5] n valeurs
F=np.frompyfunc(f,1,1) #type Object
y=F(x)+np.random.normal(0,5,n)
data=[[x,y] for x,y in zip(np.asarray(x[0]),np.asarray(y[0]))]
with open('data/secret.csv','w',newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows([['x','y']])
    writer.writerows(data)
```

Saurez-vous retrouver le code secret ?



Consignes

- Représenter les données
- Utiliser la bibliothèque `TensorFlow` pour réaliser une descente de gradient afin réaliser une régression polynômiale.

Aide

- Quelle fonction de perte utiliser ?
- Ne pas oublier de n'utiliser que des fonctions `TensorFlow` pour utiliser le ruban de gradient.
- Pour changer les donner vous pouvez utiliser la fonction `read_csv` de la bibiothèque `pandas`.
- — Pour charger tensorflow `import tensorflow as tf`
  — Les valeurs x, y sont à déclarer comme tenseur constant `tf.constant`
  — Pour calculer l'image d'un élément de x, vous pouvez utiliser a fonction `tf.math.polyval` de la bibiothèque `TensorFlow`.

1.2 Visualiser la descente de gradient

Dans cette seconde partie, vous allez visualiser la qualité de l'apprentissage de votre algorithme, grâce à TensorBoard.

1. Utilitaires
   À l'aide des bibliothèques `time` et `os`, écrire un morceau de code `python`, qui crée un répertoire nommé `logs` dans le répertoire courant s'il n'existe pas déjà.
   puis une fonction dont le prototype est le suivant :

```python
def get_logdir()->str:
    """Renvoie une chaîne de caractère <<unique>> qui
       dépend du temps à la seconde près"""
```

2. Le module `tf.summary`
   Utiliser la fonction `create_file_writer` du module `tf.summary` en conjonction avec votre fonction `get_logdir` pour créer un « écrivain ». Cet objet vous permettra d'écrire des journaux exploitables avec `TensorBoard`.
   En vous aidant de la documentation de `tf.summary`, modifier le code écrit dans la partie 1 pour remplir un journal avec les valeurs de la fonction de perte.
3. `TensorBoard` Lancer `TensorBoard` , soit dans un terminal dans le même environnement que `TensorFlow` ou depuis `JupyterLab` (cf. Les commandes magiques) :

```
%tensorboard --logdir logs
```

   Exécuter plusieurs fois votre algorithme et comparer les courbes d'apprentissages. Résultat attendu :

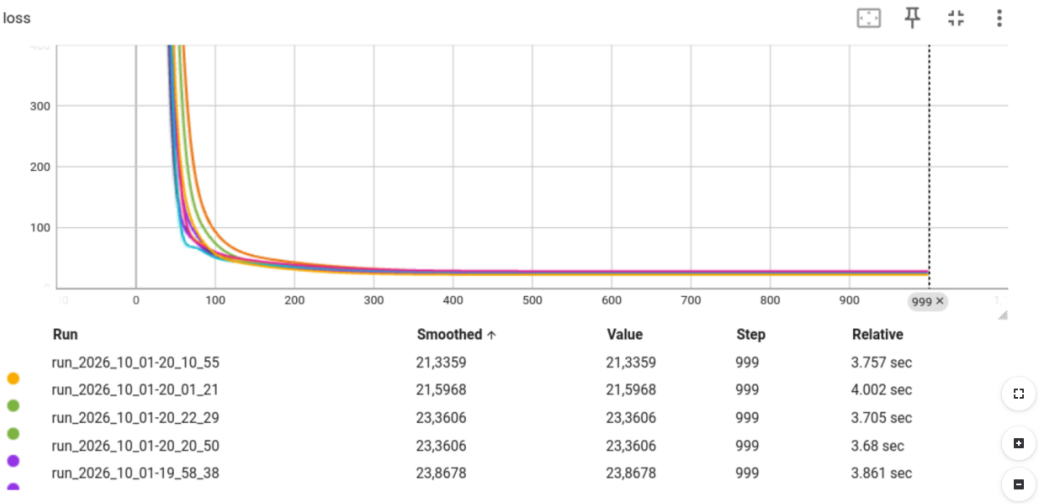


Figure 1 – Courbes d'apprentissage

2 Réseaux convolutifs

2.1 Le problème MNIST.

Selon Wikipedia,

> La base de données MNIST pour Modified ou Mixed National Institute of Standards and Technology, est une base de données de chiffres écrits à la main. C'est un jeu de données très utilisé en apprentissage automatique.

Le code suivant, s'attaque au problème de la reconnaissance de caractère manuscrit en utilisant cette base de donnée pour l'apprentissage.

```
#Chargement des biblothèques
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models
import os

# Paramètres
## Taille des mini-lots
batch_size=32
## Nombre d'époques
epochs=10
## Taille du vecteur d'entrée
input_shape = (28, 28, 1)



# Récupération des données à l'aide de tensorflow dataset

(train_ds, test_ds), ds_info = tfds.load(
     'mnist',
     split=['train', 'test'],
     as_supervised=True,
     with_info=True,
)

# Taille des mini-lots

batch_size=32
# Préparation des données

def normalize_img(image, label):
     image = tf.cast(image, tf.float32) / 255.0  # Normalisation [0, 1]
     return image, label

train_ds = train_ds.map(normalize_img).cache().batch(batch_size).prefetch(tf.data.AUTOTUN
```

```
test_ds = test_ds.map(normalize_img).batch(batch_size).cache().prefetch(tf.data.AUTOTUNE)


# Création du modèle

model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 classes pour les chiffres 0-9
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

_ = model.fit(
    train_ds,
    validation_data=test_ds,
    epochs=epochs,
    shuffle=False, #Inutile ici, car le mélange a déjà été effectué
    )


# Évaluation du modèle

score = model.evaluate(test_ds, verbose=0)

print(f"Précision : {score[1]}, fonction de perte : {score[0]}")
```

1. Exécuter ce code. Quel est le taux de précision obtenu ?
2. Expliquer une à une les lignes 41 à 49
3. À quoi servent les lignes 44 et 46 ?
4. Exécuter le morceau de code suivant, expliquer la différence avec le résultat précédent.

```
score = model.evaluate(train_ds, verbose=0)
print(f"Précision : {score[1]}, fonction de perte : {score[0]}")
```

5. Visualisation avec TensorBoard
   À l'aide de la fonction `keras.callbacks.TensorBoard` créer une fonction de rappel nommée `rappel_tensorboard` qui permet de visualiser la fonction de perte et la précision dans `TensoBoard`. (On pourra réutiliser la fonction `get_logdir`) Pour se servir de cette fonction de rappel :

```
_ = model.fit(
     train_ds,
    validation_data=test_ds,
    epochs=epochs,
    shuffle=False, #Inutile ici, car le mélange a déjà été effectué
    callbacks=[rappel_tensorboard],
    )
```

6. Arrêt précoce
   Pour éviter d'attendre la fin d'un apprentissage qui ne progresse plus, on peut utiliser une fonction de rappel (`keras.callbacks.EarlyStopping`) qui arrête l'apprentissage dès qu'un certain paramètre, par défaut la fonction de perte de l'ensemble de validation, ne progresse plus. Mettez en place cette fonction de rappel.
7. Tester votre algorithme avec un échantillon de votre écriture. Détailler votre démarche.

2.2 Cats and Dogs

Utilisez les notions vues précédemment, sur l'ensemble de données Cats vs Dogs.

— Charger les données
— Représenter quelques images de l'ensemble de données
— Augmenter l'ensemble de données
— Créer un réseau de neurones et réaliser son apprentissage.
— Évaluer les performances de votre réseau.